

Session_14 activities worksheet

Online handout: *Excerpt from Binder/Heermann*

In this Session, we'll extend our study of Monte Carlo methods. We'll compare importance sampling to ordinary random sampling and explore the two-dimensional Ising model. **Please read the background notes for an introduction and then use the Binder/Heermann handout as you go.**

A) Overview: Simulation of 2D Ising Model [HTML5 version]

To get started, we'll look at an applet that simulates the 2D Ising model. The goal is to visualize before looking at code; it is NOT expected that you will understand all of the physics now.

1. Go to the HTML5 Ising model simulation linked on Carmen and read the short introduction.
2. You will see a grid of blue and yellow squares, which represent up or down spins on a lattice. The magnetization is proportional to the net spin (count blue as +1 and yellow as -1). Below the grid you can read off the temperature and control it with a slide. There are also controls for the size of the grid and the steps per frame. Set the temperature to 10 by moving the slide all the way to the right and press start? *What are the characteristics of the system? Are there any large areas of all blue or all yellow? How large is the absolute value of the magnetization per spin (roughly)? Is it constant?*

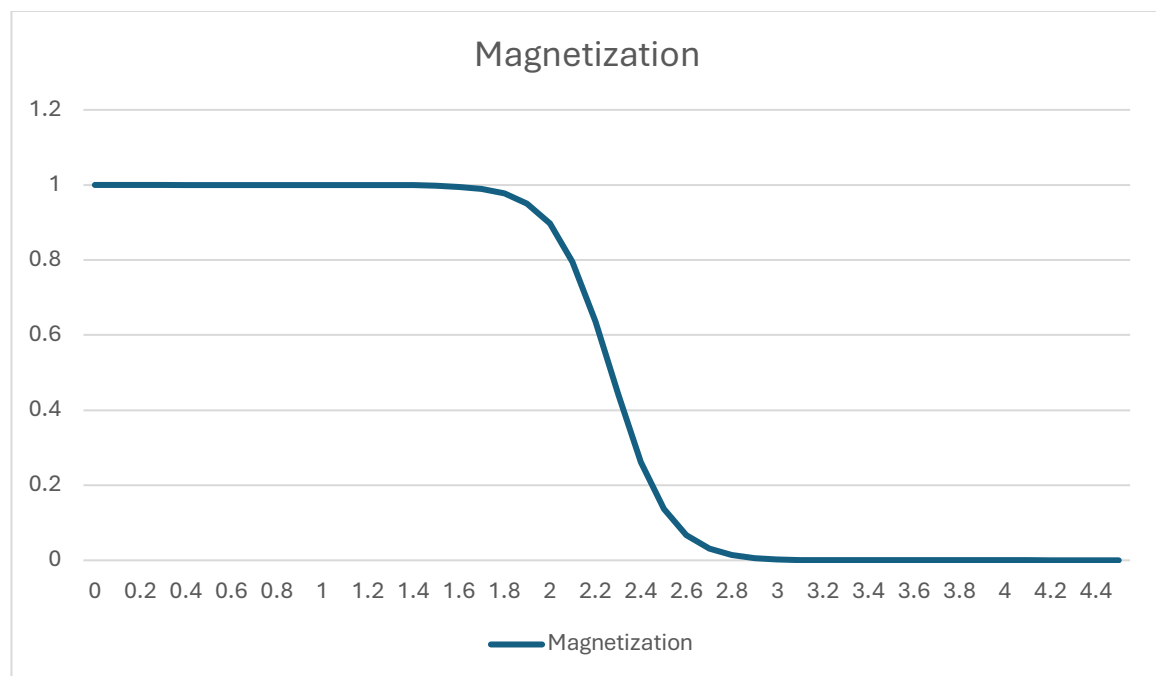
→ The characteristics of this system with $T=10$ is that it acts like “white noise” with all of the spins being randomly aligned and fluctuation with no clear pattern or order. This means there are no large areas of all blue or all yellow. The rough absolute value of the magnetization should be close to 0, since about 50% of the spins are up and 50% are down. It is not technically constant, as it is fluctuating as the spins flip, but it should stay close to 0.

3. Now rapidly bring the temperature to zero and let it settle down. Repeat the cycle of rapid heating and cooling several times. *Does it always end up in a uniform ferromagnetic state? If not, why not? Estimate the absolute value of the magnetization per spin in each case.*

→ Sometimes the system ends up in a uniform ferromagnetic state, but not always. I even found a case where we end up with two spin down vertical bands on the sides, with another vertical up band in the center that stays constant. I believe it doesn't always end up in this uniform state because as the system is cooled instantly, it forms clusters of blue and yellow. As $T \rightarrow 0$, there is no longer enough energy to flip a spin stuck on a boundary, meaning we reach some local minimum energy, instead of the global minimum. The magnetization in a uniform state should be 1, but in some sort of striped domain like the one I found it could vary depending on the size of the bands. The one I found, the total number of up and down spins seems roughly the same, so the magnetization should be close to 0, but I guess if you get one color being dominant, you could get it being close to 0.5.

4. Next change the temperature by increments of about 1 (you have to release the slide for the change to take effect). *Characterize the behavior of the system as it heats through the critical temperature to high temperature. What happens at 2.27 (restart the page to get there exactly)? Make a sketch of the absolute value of the magnetization as a function of temperature.*

→ At low temperature, the system is mostly one color, with small clusters of the opposite color appearing but then quickly vanishing, meaning the system stays relatively stable and aligned. As you increase the temperature, the large structures break down and you see “noisy” behavior of fluctuating, roughly equal amount of both spins. As we reach 2.27 (criticality), we start seeing clusters of all sizes and larger fluctuations of both spins, and the solid blocks of spins start dissolving into “flickering” spin fluctuations. An example sketch is shown below for what the magnetization vs. T would look like the following, but with more variation at the $T \rightarrow 0$ end (left side).



5. Play with changing the cell size (by the Size pull down) to very large and very small. *What differences do you observe in the size of the fluctuations and the behavior of the magnetization?*
- At small grid sizes, the magnetization is pretty “jumpy”. Due to the small number of spins, a single spin flipping massively impacts the average, so the magnetization seems to swing more often. On a large grid, the magnetization is much more stable and smooth, so while hundreds of spins are flipping, they tend to average out, so the magnetization value will move more slowly. On the small grid, the fluctuations are large and frequent, meaning the system can flip its entire state from all up to all down, whereas on the larger grid, the fluctuations may be locally large but globally small.
-

B) Monte Carlo Sampling

In the `sampling_test.cpp` code, three distributions of energy for a one-dimensional Ising model [equation (2.1.1) in the handout] are generated. The first is the exact distribution at temperature kT in the canonical ensemble; that is, the distribution of energies considering every possible configuration of spins weighted by a normalized Boltzmann factor. The second is the energy distribution if a large number of spin configurations are chosen at random. The third is the energy distribution from a Metropolis Markov process.

1. Here are some questions to get you familiar with the model and its implementation in the code `sampling_test.cpp`:
 - *How does the "J" in the Ising model determine whether we will have a ferromagnet or an anti-ferromagnet? Which choice does the code start with? What does the ground state (lowest energy state) of a ferromagnet look like? [Bonus: What is the physical origin of J for a ferromagnetic material?]*
→ The J represents the exchange energy (via the handout). When $J > 0$, the energy is lower when the spins are the same sign, so all the spins point in the same direction (ferromagnet). With $J < 0$, we then get an anti-ferromagnet, as the energy is lower when we have alternating signs. The code starts with a ferromagnet, $J=1$. This equates to a ground state with a degeneracy of 2: all spins up or all spins down. The physical origin of J comes from exchange interaction, coming from the fact the spin-particle fermions must follow the Pauli Exclusion Principle, and that electrostatic repulsion is reduced by this requirement to stay further apart. Thus, if the “energy savings” from this reduced repulsion is greater than the energy cost to align the spins, the material becomes ferromagnetic.
 - *How would you add an external magnetic field H_{ext} to the code? [See Eq.(14.6) in the notes for the Hamiltonian.]*
→ You would add some constant for the external magnetic field H_{ext} , and then update the `calculate_energy` function to include the $J*s*s$ term and the $H_{ext}*s$ term.
 - *How many total "microstates" are there for a 1-D Ising model with the number of sites in the code? (I.e., how many possible "configurations" of spins are there?) If calculating the energy of each configuration takes the same amount of time, how much longer would it take to calculate a lattice with twice the number of sites?*
→ Each site has two possible states: spin up or spin down. The code has $N=20$ sites, so the total number of possible microstates is $2*2*2*... 20$ times, or 2^{20} . If you double the total number of sites from N to $2N$, then the new total microstates goes from 2^N to $2^{(2N)} = (2^N)^2$. Thus, the calculation time would increase by a factor of 2^N by doubling the total number of states.

- *Given a number of lattice sites, what are the values of the minimum and maximum energies? How many different energy values are possible? (Hint: You can't get every integer energy between the minimum and maximum.)*
 → With $J=1$ and $N=20$, the minimum energy is the ferromagnetic ground state, $E = -1*(1+1+\dots+1) = -N = -20$. The maximum energy will be the anti-ferromagnetic state, $E = -1*(-1-1-\dots-1) = N = 20$. Since we can't get all the integer values (since flipping one spin aligned with both of its neighbors changes two $+1$ bonds to two -1 bonds), that changes the sum of s_i*s_j to $-2-2=-4$. Due to the $-J$ term, the energy increases in steps of $4J=4$. So, the possible energy values are $\{-20, -16, -12, -8, -4, 0, 4, 8, 12, 16, 20\}$ or 11 possible values.
- *What are the boundary conditions for the line of spins? Find where it is used in the code. What other set of boundary conditions could be used?*
 → The boundary conditions for the line of spins is periodic, treating the 1D line like a closed loop. This is found in the `calculate_energy` function after the loop. You could use other boundary conditions, like a free boundary condition, simply ending the line like a finite wire. We could also use ghost boundary conditions that lock some spins at permanent specific state positions.
- *Find the calculation of the exact partition function in the code. How is this equal to the same result as (2.1.5) in the Binder/Heerman notes? [Hint: You can add $\exp(-E_i/kT)$ for each spin configuration separately, or group all those with the same energy together and multiply by how many have that energy.]*

→ The calculation of the exact partition function in the code is at the comment stating “find the partition function at temperature kT ”, with the variable `Z_exact`. The code takes the sum of the energy count over some bin i (call it $f(E_i)$) times the Boltzmann factor, $f(E_i)*e^{(-E_i/kT)}$. The equation 2.1.5 in Binder/Heermann is an integral over dx of $\exp(-H(x)/kT)$. For a discrete system, we can easily transform that into the sum of $\exp(-H(x)/kT)$, and here $H(x)$ is just the energy of the state, so this would be the sum of $\exp(-E_s/kT)$, where s is the state. So, we have two methods of calculating the partition function: Binder/Heerman sums over each Boltzmann factor for each individual state, while the code sums of exponentials for a specific energy value (or bin) E_i . So, we can write this mathematically as the following expression, seeing it's the same (using $\beta=1/kT$). The left hand side is Binder/Heerman, while the right-hand side is what we do in the code.

$$\sum_{\text{states } s} e^{-\beta E_s} = \sum_{\text{unique energy } E} \left(\sum_{\text{states with energy } E} 1 \right) e^{-\beta E}$$

Can you explain how the complete set of configurations are constructed in the code using the "next_configuration" function?

→ The `next_configuration` function uses binary to treat each spin as a bit and then get the state. So, for a two-site system, 00 can be both down, 01 is first down second up, 10 is first up second down, and 11 is both up (The code actually uses +1 and -1, but the 1 and 0 is easier to explain since it's directly binary). The code then processes the spins by flipping the spin from -1 to +1 or +1 to -1. It then ends the recursion once we reach the very last site.

2. Compile and link `sampling_test.cpp` (using `make_sampling_test`) and run it to see what the output looks like. *Did you get the correct answer above for the number of configurations and the possible energies? (If not, rethink!)*

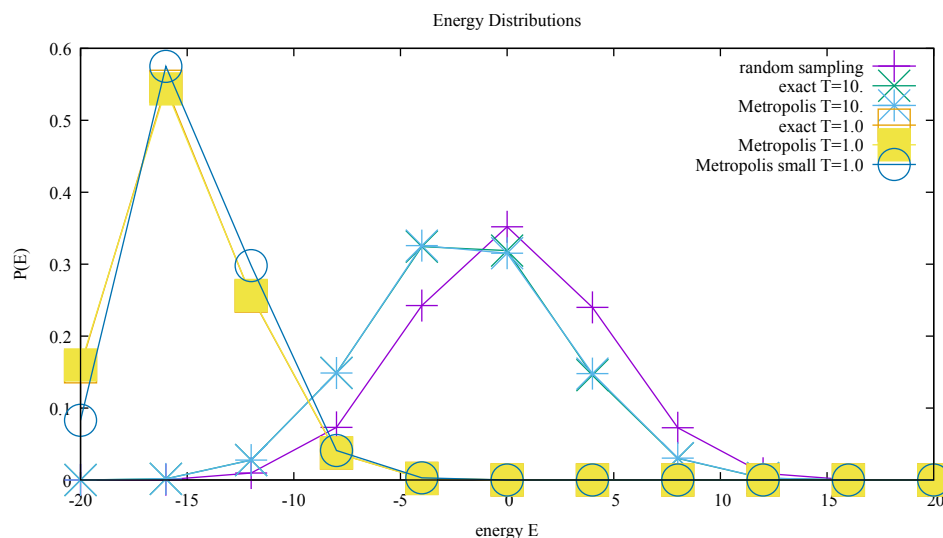
- Look through the code and try to understand how each distribution is generated.

→ I did get the correct number of configurations and possible energies! I do believe I understand how each distribution is generated.

3. Generate and attach a gnuplot graph of the probability of energy E [called $P(E)$] vs. the energy (this is what is output to the screen) for $kT = 10$ and $kT = 1$. [You'll need to get the results into data files somehow. There is a plot file `sampling_test.plt` to help with the plot, but you have to put the results in files with these names. Cutting and pasting is fine.]

- Why is the distribution for ordinary random sampling centered at $E=0$? Based on this distribution, what kind of temperature does ordinary random sampling correspond to? [Hint: What Boltzmann factor would result in a distribution like this?]

→ The plot is below. The ordinary random sampling ignores the temperature kT or the Boltzmann factor, simply picking spin configurations at random, so it's like flipping a random fair coin, which will center itself around 0. This would be like an infinite temperature (or at the very least, a very large temperature), as for high temperatures, the Boltzmann factor approaches 1, and then every configuration has the same weight, like an ordinary random sampling.



- Compare the exact $P(E)$ for a canonical distribution to those of random sampling and the Metropolis algorithm. *Will the Metropolis algorithm be effective in doing importance sampling (which concentrates points where the integrand is large)? Why will the random sampling be a problem for evaluating thermal averages [see figure 2.3 and the discussion after equation (2.1.33) in Binder/Heermann]?*

→ The Metropolis algorithm appears to be effective in doing importance sampling, as the Metropolis data fits the exact curves very well (even with low steps in “Metropolis” small). In contrast, random sampling will be problematic for evaluating thermal averages because it ignores the temperature and samples based solely on the density of the states, which peaks at $E=0$. As a result, random sampling basically spends all its effort in high-energy regions that may have much lower probabilities (like with $T=1$), while the Metropolis algorithm focuses on the “important” energy states that actually contribute more to the average.

4. Verify that the transition probabilities in (2.1.39a) and (2.1.39b) both satisfy the condition (2.1.38). *Which one is implemented in the code?* Bonus: Switch to the other and check that it works.

→ I have verified the transition probabilities satisfy 2.1.38. It appears 2.1.39b is the one in the code, as I see `delta_energy>0` as an option. I switched to 2.1.39a, and it does appear to work too.

5. *How do you calculate the average energy at a given temperature? Modify the code to calculate the average energy at $kT = 1$. and $kT = 10$. using the Metropolis sampling methods and compare to the exact average energy (according to the canonical Boltzmann distribution).*

→ The average energy $\langle E \rangle$ can be calculated by the sum of $E_i * P(E_i)$. For $kT=1$, we get that the exact average energy is -15.28 and Metropolis gets -15.03. For $kT=10$, we get exact energy average as -1.993 and Metropolis gets -1.992.

C) The Two-D Ising Model

In this section, we explore some aspects of Monte Carlo simulations that are discussed in section 2.2 of the handout. We use the two-dimensional Ising model first with a ferromagnetic interaction ($J > 0$) and then with an "anti-ferromagnetic" interaction as our example.

1. Take a look at `ising_model.cpp` and note that one can switch between the one-d and two-d Ising models by commenting and uncommenting some lines. The default is the two-d model. *What are the differences between the one-d and two-d versions?*

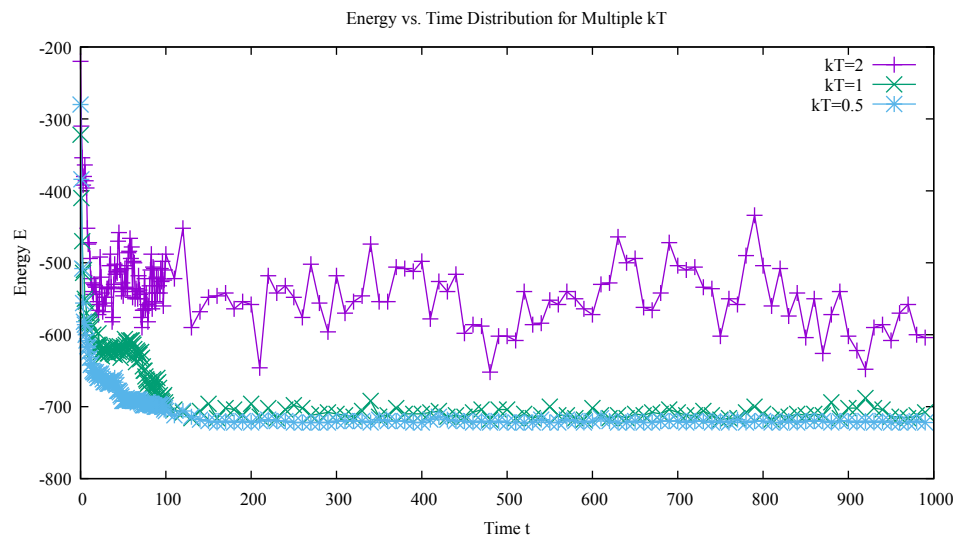
→ There are some differences. The number of `linear_sites` is different (N for 1D, $N*N$ for 2D). The energy calculation is handled differently too. In 1D, each spin has two neighbors (left and

right), while in 2D there are four (up, down, left, right). Also it seems that the 2D uses free boundary conditions instead of periodic.

2. **Equilibration.** Compile and link `ising_model.cpp` (use `make_ising_model`) and run for several temperatures.

- Make a gnuplot graph of the energy vs. time for $kT = 2.0$, 1.0 , and 0.5 (all on the same graph, so rename your files appropriately). *Run repeatedly at $kT = 1.0$ until you see that you don't always get the same answer at large times. What does this mean?*

→ I do see that we don't get the same answer at large times. I believe this is because the Metropolis algorithm relies on random sampling, causing unique runs each time. At lower temperatures, maybe the system gets stuck in a local minimum for energy, which likely means that the simulation does not reach thermal equilibrium. So, we probably need a longer equilibration period or average over multiple runs to get a more stable result. But, looking at the plot, I'm more inclined to believe random variation than not reaching thermal equilibrium. The plot is below:



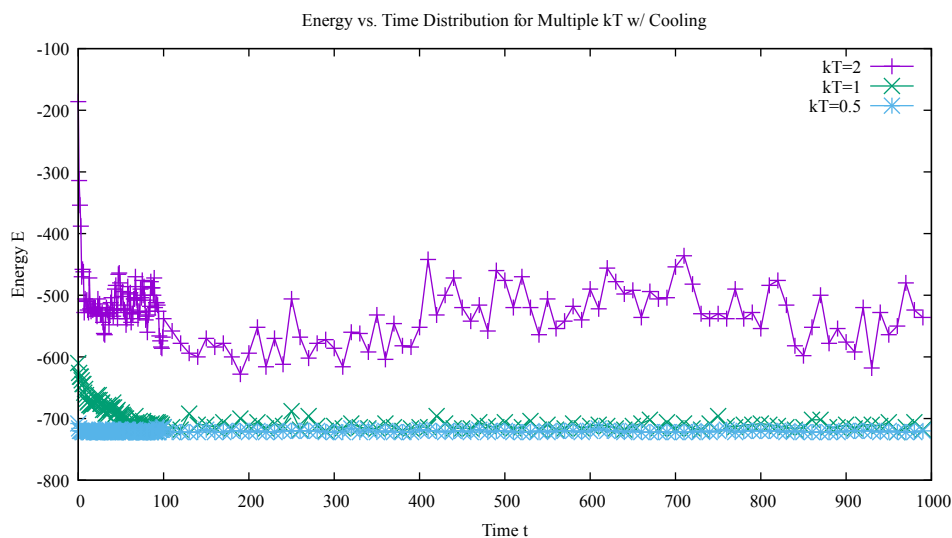
- Look at the small time region. *How long does it take at each temperature to reach "equilibrium"? If you were going to use the configurations generated here to calculate thermal averages, would you want to use the configurations at the beginning? How can you deal with this?*

→ It looks like it takes $kT=0.5$ around 250-300 steps to reach equilibrium, $kT=1.0$ is around 150-200 steps, and $kT=2.0$ is around 20-50 steps (but has large fluctuations throughout the entire run). Because the system is settling near the beginning, you don't want to use the configurations near the beginning to calculate averages. You can deal with this by implementing a "burn-in" by discarding initial data points to some fixed time and then deal with fluctuation by averaging over many steps (or perform multiple independent runs).

- BONUS: Modify the code so that the output file names automatically have the temperature in their names. (Recall filename_test.cpp from an earlier Session.)
→ Done! (I actually did that earlier instead of manually changing the names).
3. Modify the code to change it from ferromagnetic to anti-ferromagnetic (only one line needs to be changed!). *What did you change?*
→ You just need to change J_{ising} from 1.0 to -1.0.
 4. **Cooling.** At present, the code starts from a random configuration. Modify the code to implement "cooling" by looping through temperatures $kT = 2.0, 1.0$, then 0.5 but start the simulation at each successive temperature using the final configuration of the higher temperature as the initial configuration of the lower temperature. *Generate a gnuplot graph of the energy vs. time for each of the temperatures and compare to your previous results.*

Conclusions?

→ Below is the plot. What we see is that we have definitely eliminated or reduced the initial transients, reducing time to equilibrate. We see larger fluctuations at $kT=2$, but decreasing temperature decreases these fluctuations, settling the energy in a tighter, lower band. This allows us to get closer to its minimum possible energy with $kT=0.5$, helping guide the system into its ground state.



Wed Apr 08 14:28:56 2026

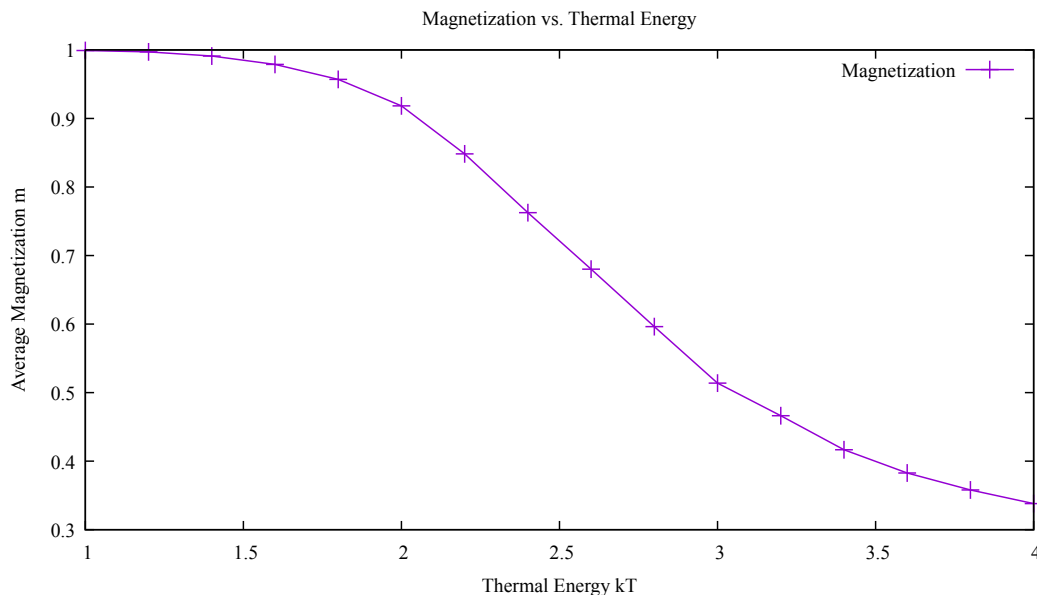
5. **Efficiency.** The code at present has several inefficiencies. Here are some ways to speed it up:
 - The current approach compares energies of new and old configurations by calculating the full energy of each and subtracting. *Can you devise a (much) more efficient approach?* (You don't need to implement it here.)
→ I believe the inefficiencies come from the total calculation of the lattice. It would be easier to just calculate the change in energy associated with flipping a single spin at some site, which only depends on its nearest neighbors. So, you would calculate ΔE which would be only based on the nearest neighbors.

- During one MCS, we can update each spin sequentially, which saves random number calls but also leads to shorter equilibration times.
- We can reduce the random number calls when deciding if a spin flips or not.
- We can use a table of nearest neighbors of each spins to save calculation time.

These optimizations speed up the code by a factor of about six! A new version is `ising_opt.cpp` (with `make_ising_opt`). Look at how the optimizations were implemented.

6. Phase Transition

- Modify the optimized code `ising_opt.cpp` to calculate the absolute value of the magnetization of the system (with linear size $L=5$) for various temperatures. Starting at $kT=4.0$, cool down the system by $\Delta kT = 0.2$ until $kT=1.0$. *What do you observe about the behavior of magnetization?* (Make a plot.)
 → We see at low temperatures, the magnetization (and thus spins) are “frozen”, meaning that the spins are aligned. Once we pass the critical temperature, the magnetization drops to 0, showing the fluctuations or even numbers in the spins. Due to the low linear sites being used, this decrease is less sharp and not as pronounced. The plot is below:



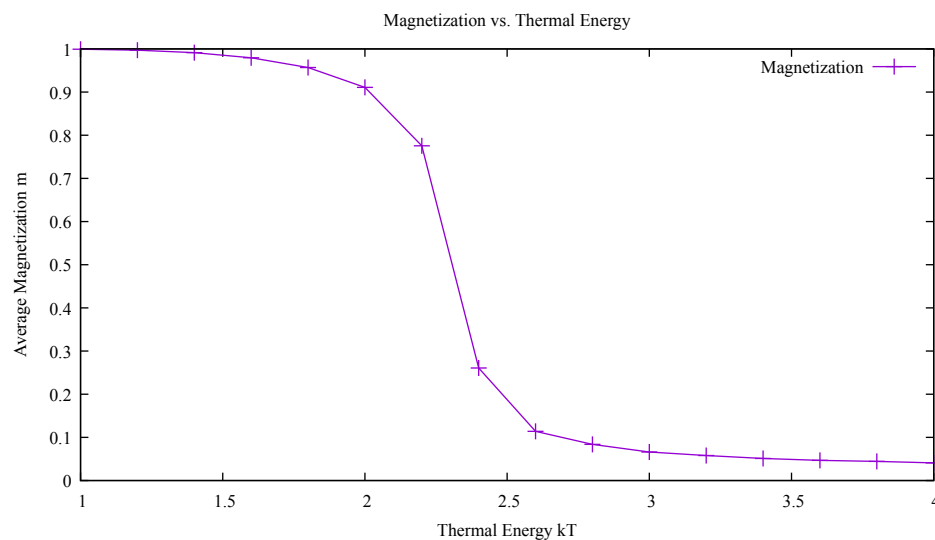
Fri Apr 10 13:18:56 2026

- *When we calculate the magnetization, we use the absolute value of it, why?* It may help to look at the time dependence of the magnetization around $kT=2$ with size $L=5$ or less. [Bonus: Can you explain your observation in terms of "spontaneous symmetry breaking"?] Will this problem get better or worse when you increase the system size? [Hint: Would it be possible for an infinite system to change from a state with all up spins to one with all down spins?]
 → We use the absolute value because if we try to average over some time period $\langle M \rangle$, the system's flipping in $+1$ and -1 states will cancel each other out, yielding $\langle M \rangle \rightarrow 0$. By taking the absolute value, we can prevent that, allowing us to identify the phase transition based on relative alignment. In terms of spontaneous symmetry breaking, at high temperatures the system is disordered with no preferred direction. As we cool the

down, the system chooses one of the two ground states (all up or all down) spontaneously. This also works when heating from a cold state, with all spins aligned breaking into disordered fluctuations. This is spontaneous symmetry breaking, and this is the switch we see at the critical temperature from order to disorder and vice versa. The problem should get better with increased system size, because for an infinite system, it becomes mathematically impossible for the system to completely flip its global magnetization from all up to all down or the other way around.

- Now change the system size (try $L=10, 20, 40$). *By changing the system size, can you observe a change in the behavior of magnetization? If so, can you make an argument why this happens?*

→ As we increase the system size, we see an increase in the sharpness of the phase transition, meaning the transition becomes much more abrupt. As we increase the system size, it becomes more difficult to flip from an all up state to an all down state (or vice versa). With more lattice sites, the energy barrier required to flip more sites into a unified spin state grows, making the ordered state more stable against spontaneous fluctuations. This means that the graph should stay relatively stable until the critical temperature is reached, where it “jumps” to disorder (or jumps to order if going the other direction). Below is a graph of $L=40$:



Fri Apr 10 13:28:24 2026

D) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→ This worksheet taught me how to write correct clear code, and I had a better understanding of physics through putting physics concepts into code, especially with magnetization and the intuition behind it. I also now have a better understanding of physics through playing with numerical solutions, and the HTML visualization helped a lot with understanding the physics and the code too.